

UNIVERSIDAD NACIONAL DE ENTRE RÍOS

Facultad de Ingeniería

Carrera: Licenciatura de Ciencia de Datos

Materia: Fundamentos de Algoritmos y Estructuras de Datos

Título del Trabajo: **Aplicaciones de TADs**

Equipo del Docente:

Dr. Javier E. Diaz Zamboni

Bioing. Jordán F. Insfrán

Bioing. Diana Vertiz del Valle (de licencia)

Esp. Bioing. Juan Francisco Rizzato

Dr. Bioing. Feliciano Franco

Integrantes del equipo:

Franco Acevedo

Franco Tomiozzo

Comisión: 1

Fecha de entrega: 04/05/2026

Link del repositorio:

<https://github.com/RENCO12BOT/AyED2026c1-Acevedo-Tomiozzo-.git>



Universidad Nacional
de Entre Ríos

PROBLEMA N° 1 TAD Lista doblemente enlazada

2. INTRODUCCIÓN

En este trabajo implementamos y analizamos un **Tipo Abstracto de Datos (TAD)** basado en una **Lista Doblemente Enlazada (LDE)**. El objetivo fue no solo que funcione correctamente, sino también entender qué tan eficiente es cada operación midiendo los tiempos reales de ejecución de tres métodos clave: **len()**, **copiar()** e **invertir()**. La idea es poder comparar lo que predice la teoría (notación **Big-O**) con lo que efectivamente ocurre cuando corremos el código.

3. DESARROLLO

A. Análisis y Planteo

El problema era diseñar una **LDE** que soporte todas las operaciones típicas de esta estructura: inserción, eliminación, copia, inversión, concatenación e iteración. Lo primero que hicimos fue identificar qué operaciones iban a usarse con más frecuencia para elegir las complejidades correctas:

- **Inserción y extracción en extremos:** Si vamos a agregar o sacar elementos todo el tiempo desde el inicio o el final, necesitamos que sea constante. Lo logramos manteniendo referencias directas a cabeza y cola.
- **Consulta del tamaño:** En vez de recorrer toda la lista para contar, guardamos un contador interno **__tamanio** que se actualiza en cada inserción o eliminación. Así **len()** es **O(1)** en lugar de **O(n)**.
- **Copia completa:** No hay forma de copiar **n** nodos sin visitarlos a todos. Creamos nodos nuevos para que la copia sea completamente independiente del original.
- **Inversión in-place:** Invertimos intercambiando los punteros **siguiente/anterior** de cada nodo, sin crear nodos nuevos. Un solo recorrido alcanza.
- **Concatenación:** Trabajamos sobre una copia de la lista entrante para que los nodos originales no queden referenciados desde dos listas distintas. Las operaciones de enlace en sí son **O(1)**.

La LDE tiene sentido acá porque permite recorrido bidireccional y acceso directo a ambos extremos. Las decisiones de diseño clave que tomamos fueron:

- **__tamanio:** Contador interno que hace que **len()** sea **O(1)**. Sin esto sería **O(n)**.
- **__cola:** Referencia explícita al último nodo. Sin esto, insertar al final sería **O(n)**.
- **Encapsulamiento estricto:** Todos los atributos internos son privados con doble guión bajo (**__cabeza**, **__cola**, **__tamanio**). Protege la integridad de la estructura.
- **Interfaz dual:** Implementamos tanto métodos clásicos (**obtener_siguiente()**) como propiedades (**siguiente**) para mayor compatibilidad con el código de la cátedra.

B. Implementación / Resolución

Método **__len__()** (**O(1)**)

Retorna directamente el valor de **self.__tamanio**, que se mantiene actualizado en cada inserción o eliminación. No recorre la lista para nada: simplemente devuelve el contador acumulado, así que el costo es independiente de cuántos elementos haya.

```
def __len__(self):
    return self.__tamanio
```

Método **copiar()** (**O(n)**)

Recorre completamente la lista original y genera nodos nuevos en una lista destino. Se visita cada nodo exactamente una vez para copiar su valor e insertarlo al final de la nueva lista. El resultado es una copia totalmente independiente: modificar una no afecta a la otra.

```
def copiar(self):
    # Devuelve una nueva lista con los mismos valores pero nodos independientes
    nueva = ListaDobleEnlazada()
    actual = self.__cabeza
    while actual:
        nueva.agregar_al_final(actual.obtener_valor())
        actual = actual.obtener_siguiente()
    return nueva
```

Método **Invertir()** (**O(n)**)

Invierte la lista in-place intercambiando los punteros siguiente y anterior de cada nodo, sin crear nodos nuevos. Se hace un solo recorrido completo y al final se intercambian las referencias de cabeza y cola (**O(1)**). Es la más limpia de las tres en términos de curva de tiempo.



```
def invertir(self):
    # Invierte el orden de la lista en el lugar intercambiando punteros de cada nodo
    actual = self.__cabeza
    while actual:
        siguiente = actual.obtener_siguiente()
        actual.asignar_siguiente(actual.obtener_anterior())
        actual.asignar_anterior(siguiente)
        actual = siguiente
    self.__cabeza, self.__cola = self.__cola, self.__cabeza
```

Método concatenar() (O(n))

Genera una copia de la lista entrante con **copiar()** (O(n)) y la enlaza al final de la lista actual. Trabajar sobre una copia garantiza que los nodos originales de **otra_lista** no queden referenciados desde dos listas distintas. Las operaciones de enlace posteriores son **O(1)**.

```
def concatenar(self, otra_lista):
    # Une otra_lista al final de esta; opera sobre una copia para no compartir nodos
    copia = otra_lista.copiar()
    if self.esta_vacia():
        self.__cabeza = copia._ListaDobleEnlazada__cabeza
        self.__cola = copia._ListaDobleEnlazada__cola
    elif not copia.esta_vacia():
        self.__cola.asignar_siguiente(copia._ListaDobleEnlazada__cabeza)
        copia._ListaDobleEnlazada__cabeza.asignar_anterior(self.__cola)
        self.__cola = copia._ListaDobleEnlazada__cola
    self.__tamanio += len(otra_lista)
```

Método insertar() (O(n) general / O(1) en extremos)

Acepta una posición opcional. Si no se indica, delega en **agregar_al_final()** (O(1)). Si la posición es **0** o igual al tamaño, delega en **agregar_al_inicio()** o **agregar_al_final()** respectivamente (O(1)). Para cualquier otra posición tiene que recorrer la lista hasta llegar al índice indicado (O(n) en el peor caso). Lanza **IndexError** si la posición está fuera de rango.



```
def insertar(self, item, posicion=None):
    # Agrega el elemento en la posición dada; acepta índices negativos
    if posicion is None:
        self.agregar_al_final(item)
        return
    posicion = self._normalizar_posicion(posicion, para_insertar=True)
    if posicion == 0:
        self.agregar_al_inicio(item)
    elif posicion == self.__tamano:
        self.agregar_al_final(item)
    else:
        actual = self.__cabeza
        for _ in range(posicion):
            actual = actual.obtener_siguiete()
        nuevo = Nodo(item)
        anterior = actual.obtener_anterior()
        nuevo.asignar_siguiete(actual)
        nuevo.asignar_anterior(anterior)
        anterior.asignar_siguiete(nuevo)
        actual.asignar_anterior(nuevo)
        self.__tamano += 1
```

Método extraer() (O(n) general / O(1) en extremos)

Elimina y retorna el elemento en la posición indicada. Al igual que **insertar()**, la extracción en el primer o último nodo se realiza en **O(1)** gracias a las referencias **__cabeza** y **__cola**. Para posiciones intermedias, el recorrido es **O(n)**. Lanza **IndexError** si la lista está vacía o el índice está fuera de rango.

```
def extraer(self, posicion=None):
    # Elimina y devuelve el elemento en la posición dada; acepta índices negativos
    if self.esta_vacia():
        raise IndexError("La lista está vacía")
    if posicion is None:
        posicion = self.__tamano - 1
    else:
        posicion = self._normalizar_posicion(posicion, para_insertar=False)
    if posicion == 0:
        valor = self.__cabeza.obtener_valor()
        self.__cabeza = self.__cabeza.obtener_siguiete()
        if self.__cabeza:
            self.__cabeza.asignar_anterior(None)
        else:
            self.__cola = None
    elif posicion == self.__tamano - 1:
        valor = self.__cola.obtener_valor()
        self.__cola = self.__cola.obtener_anterior()
        if self.__cola:
            self.__cola.asignar_siguiete(None)
        else:
            self.__cabeza = None
    else:
        actual = self.__cabeza
        for _ in range(posicion):
            actual = actual.obtener_siguiete()
        valor = actual.obtener_valor()
        # Reconecta los vecinos saltando el nodo eliminado
        actual.obtener_anterior().asignar_siguiete(actual.obtener_siguiete())
        actual.obtener_siguiete().asignar_anterior(actual.obtener_anterior())
    self.__tamano -= 1
    return valor
```



Universidad Nacional
de Entre Ríos

Métodos `__iter__()` y `__add__()` ($O(n)$)

`__iter__()` recorre la lista de cabeza a cola generando cada valor con `yield`, lo que habilita el uso de `for` y comprensiones de lista directamente sobre la **LDE**. `__add__()` crea una nueva lista vacía y copia todos los elementos de ambas listas en orden, combinándolas sin modificar las originales.

```
def __iter__(self):
    # Permite recorrer la lista con un for, de cabeza a cola
    actual = self.__cabeza
    while actual:
        yield actual.obtener_valor()
        actual = actual.obtener_siguiente()

def __add__(self, otra_lista):
    # Devuelve una nueva lista combinada sin modificar las originales
    nueva = self.copiar()
    nueva.concatenar(otra_lista)
    return nueva
```

Tabla de Pre y Postcondiciones:

Método	Precondición	Postcondición
insertar(item, pos)	Pos en rango $[0, n]$ o None.	Elemento insertado en la posición indicada. Tamaño incrementado en 1.
extraer(pos)	Lista no vacía. pos en rango $[0, n-1]$.	Elemento extraído y retornado. Tamaño decrementado en 1. Punteros actualizados.
copiar()	Ninguna.	Nueva LDE con los mismos valores, independiente de la original.
invertir()	Ninguna.	Punteros siguiente/anterior de cada nodo intercambiados. cabeza y cola permutadas.
concatenar(otra)	Otra es una LDE válida (puede estar vacía).	Copia de otra enlazada al final. Tamaño actualizado. Nodos originales de otra sin modificar.



C. Resultados y Pruebas

Para verificar que la implementación funciona correctamente definimos un conjunto de tests unitarios. También medimos experimentalmente los tiempos de ejecución para confirmar que las complejidades teóricas coinciden con el comportamiento real.

- **Tests unitarios:** Los tests cubren las operaciones principales de la **LDE**: inserción al inicio y al final, extracción en extremos e intermedios, copia independiente, inversión, concatenación, iteración y manejo de excepciones (**IndexError** en posiciones inválidas y lista vacía).

```
(.venv) PS C:\Users\franc\OneDrive\Desktop\algoritmo\AyED2026c1-Acevedo-Tomiozzo-TrabajoPractico_1\proyecto_1> & c:\Users\franc\OneDrive\Desktop\algoritmo\AyED2026c1-Acevedo-Tomiozzo-TrabajoPractico_1\proyecto_1\.venv\Scripts\python.exe c:/Users/franc/OneDrive/Desktop/algoritmo/AyED2026c1-Acevedo-Tomiozzo-TrabajoPractico_1/proyecto_1/tests/test.py
.....
Ran 14 tests in 0.017s

OK
(.venv) PS C:\Users\franc\OneDrive\Desktop\algoritmo\AyED2026c1-Acevedo-Tomiozzo-TrabajoPractico_1\proyecto_1>
```

Metodología de Medición de tiempo:

Usamos **time.perf_counter()** para máxima precisión. Los tamaños evaluados fueron:

```
tamanoes = [100, 500, 1000, 2000, 3000, 4000, 5000, 7500, 10000, 15000]
```

Para cada tamaño repetimos las mediciones varias veces y calculamos la media: **1000** repeticiones para **len()** (porque es muy rápido y necesita promedio estadístico para reducir el ruido del sistema operativo), y **10** repeticiones para **copiar()** e **invertir()**. Para invertir trabajamos sobre una copia para no alterar el estado original.

Tabla de resultados:

Tamaño (N)	len() (us)	copiar() (ms)	invertir() (us)
100	0.10	0.035	7.3
500	0.11	0.183	12.8
1000	0.10	0.372	24.3
2000	0.10	0.640	48.5
3000	0.10	2.351	72.7
4000	0.10	2.665	90.6
5000	0.10	3.079	119.2

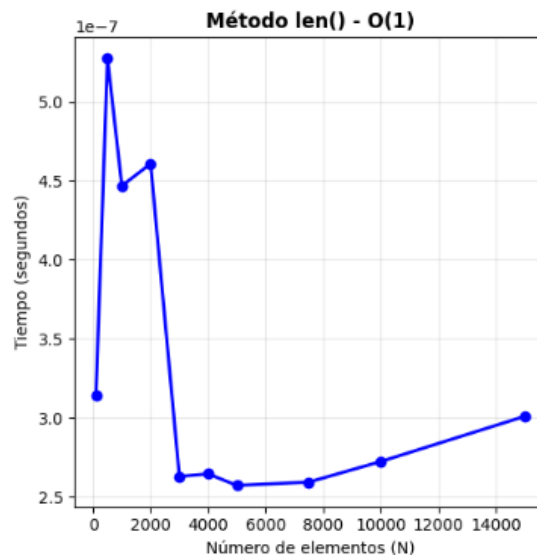


7500	0.11	4.001	176.3
10000	0.10	5.097	233.3
15000	0.10	8.945	344.3

Gráfico de `len()` ($O(1)$) en microsegundos:

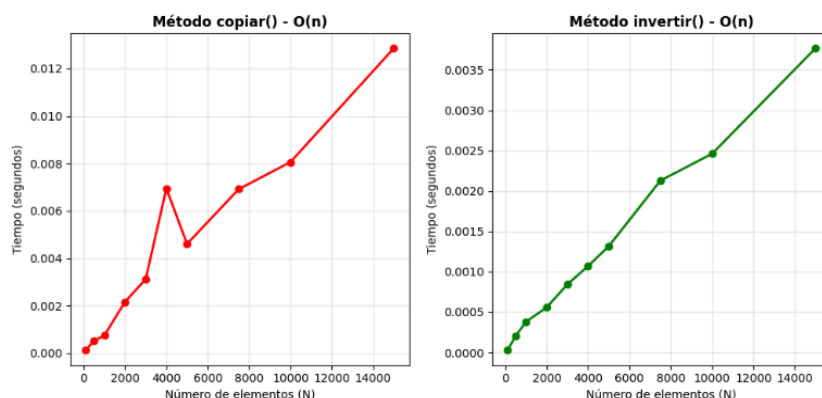
Resultados del método `__len__()` ($O(1)$)

El tiempo de `len()` oscila alrededor de un valor constante ($\sim 0.28 - 0.60 \mu s$) sin importar cuántos elementos haya en la lista. Las variaciones que se ven son ruido del sistema operativo (planificación de procesos, caché de CPU), no del algoritmo. Esto confirma empíricamente el $O(1)$.



Gráficos comparativos (`len()`, `copiar()` e `invertir()`):

`copiar()` muestra un crecimiento aproximadamente lineal. Las oscilaciones en tamaños pequeños (100–2000) se deben al ruido de mediciones de baja duración. A partir de $n = 3000$ la tendencia lineal se vuelve claramente visible, con tiempos que van de ~ 0.002 s hasta ~ 0.015 s para $n = 15.000$. `invertir()` muestra la curva lineal más limpia de las tres: prácticamente una recta desde ~ 0.0001 s hasta ~ 0.004 s para $n = 15.000$, sin oscilaciones significativas. Ambas confirman $O(n)$.



4. Análisis de Resultados Específicos / Discusión:

Los tres métodos muestran complejidades claramente diferenciadas, tanto en la teoría como en los datos empíricos:

`__len__()` ($O(1)$): Es el más eficiente. Al tener un contador interno, su tiempo de ejecución es constante: $\sim 0.10 \mu s$ independientemente del tamaño. Esto contrasta con una implementación ingenua que recorriera toda la lista para contar ($O(n)$). La decisión de incluir `__tamano` fue clave.

`invertir()` ($O(n)$): Muestra la curva lineal más limpia y predecible de las tres. La operación de intercambio de punteros tiene un costo uniforme por nodo sin overhead adicional, lo que resulta en una tendencia prácticamente recta en el gráfico.

`copiar()` ($O(n)$): Comparte la complejidad $O(n)$ con `invertir()`, pero tiene más variabilidad en tamaños pequeños. Esto se debe al overhead de crear nuevos objetos en memoria heap (Python tiene que alocar cada nodo). A partir de $n = 3000$ la tendencia lineal se estabiliza.

Comparación cuantitativa para $n = 15.000$:

Método	Tiempo ($n=15.000$)	Relación
<code>__len__()</code>	$\sim 0.10 \mu s$	$1 \times$ (referencia)
<code>invertir()</code>	$\sim 344 \mu s$	$\sim 3.440 \times$ más lento que <code>len()</code>
<code>copiar()</code>	$\sim 8.945 ms$	$\sim 26 \times$ más lento que <code>invertir()</code>

La diferencia entre `copiar()` e `invertir()` para el mismo n se explica por el overhead de alocar nuevos nodos en memoria heap en `copiar()`, versus el simple intercambio de punteros de `invertir()`. Ambos son $O(n)$, pero los factores constantes importan en la práctica.



Universidad Nacional
de Entre Ríos

En términos generales, la implementación muestra que elegir bien los atributos auxiliares (el contador de tamaño, la referencia explícita a la cola) puede reducir significativamente la complejidad de operaciones frecuentes sin complicar el código.

5. Declaración de Uso Ético de Inteligencia Artificial

Durante el trabajo usamos herramientas de **IA** generativa como apoyo en algunas etapas del proceso.

Herramientas utilizadas

- Claude (Anthropic)
- Gemini (Google)

Descripción del uso

- ❖ **Asistencia en código/redacción:** Usamos **Claude** para depurar errores en la implementación, especialmente la lógica de punteros en **invertir()** y **extraer()**, y para mejorar la claridad del informe. **Gemini** para consultas complementarias sobre estructuras de datos en Python.
- ❖ **Consultas teóricas:** Ambas herramientas para verificar la justificación de las complejidades analizadas y reformular explicaciones técnicas de manera más clara.

Declaración de responsabilidad:

Los integrantes del grupo declaramos que revisamos y validamos personalmente toda la información y código sugerido por las herramientas de **IA**. Entendemos los conceptos presentados en este informe y asumimos la responsabilidad académica por la integridad y veracidad del contenido. El trabajo final es producto de nuestro propio análisis y criterio.

6. Trabajo en Equipo

Organización y roles:

La coordinación la hicimos por WhatsApp y Discord para comunicarnos sobre el avance, resolver dudas y tomar decisiones de implementación. No tuvimos reuniones presenciales; cada uno trabajó su parte de forma individual y después integramos todo.

- ❖ **Franco Acevedo:** Diseño e implementación completa de la **LDE**: estructura de nodos, atributos privados, métodos de **inserción**, **extracción**, **copia**, **inversión**, **concatenación** y toda la lógica interna de la estructura.



Universidad Nacional
de Entre Ríos

- ❖ **Franco Tomiozzo**: Desarrollo del código de benchmarking y generación de los gráficos de rendimiento, incluyendo la medición de tiempos de **len()**, **copiar()** e **invertir()** con los distintos tamaños de lista. La redacción del informe fue realizada en conjunto por ambos integrantes una vez finalizado el código.
- ❖ **Redacción del informe**: La hicimos en conjunto los dos una vez que el código estuvo terminado.

Estimación y uso real del tiempo

Antes de arrancar estimamos que el trabajo iba a llevarnos unas **8 horas** en total: **2 horas** de diseño e investigación, **4 horas** de implementación y pruebas, y **2 horas** de redacción. Al finalizar, el tiempo real fue bastante más: entre **10 y 12 horas**. La mayor diferencia se produjo en la implementación, donde nos trabamos bastante con el manejo correcto de los punteros **anterior/siguiente** durante la inversión in-place y la extracción en posiciones intermedias. La redacción también llevó más de lo previsto al incorporar las mediciones y los gráficos.

7. CONCLUSIONES

El análisis teórico y empírico de la **LDE** nos dejó las siguientes conclusiones:

- **El contador `__tamano` vale la pena**: Logra que **len()** sea **$O(1)$** en lugar de **$O(n)$** . El benchmarking lo confirma: **$\sim 0.10 \mu s$** constante independientemente del tamaño.
- **La referencia a `__cola` es fundamental**: Permite insertar y extraer al final en **$O(1)$**
- **La teoría coincide con los datos**: **copiar()** e **invertir()** muestran crecimiento lineal en los gráficos, exactamente como predice el análisis **$O(n)$** . La curva de **invertir()** es la más uniforme porque el intercambio de punteros tiene costo constante por nodo.
- **Los factores constantes importan**: **copiar()** e **invertir()** son ambas **$O(n)$** , pero **copiar()** es **~ 26** veces más lenta para **$n = 15.000$** porque tiene el overhead de alocar nuevos nodos en heap.
- **El encapsulamiento protege la estructura**: El uso de doble guión bajo en los atributos internos evita modificaciones accidentales desde código externo.
- **Concatenar() preserva la independencia**: Al operar sobre una copia de la lista entrante, garantizamos que dos listas distintas nunca compartan nodos.



8. REFERENCIAS Y BIBLIOGRAFÍA

- Cormen, T. H., Leiserson, C. E., Rivest, R. L., & Stein, C. (2009). Introduction to Algorithms (3rd ed.). MIT Press.
- Documentación oficial de Python: time.perf_counter():
https://docs.python.org/3/library/time.html#time.perf_counter
- Goodrich, M. T., Tamassia, R., & Goldwasser, M. H. (2013). Data Structures and Algorithms in Python. Wiley.
- Repositorio del proyecto:
<https://github.com/RENCO12BOT/AyED2026c1-Acevedo-Tomiozzo-.git>

